

线性表的链式表示

简称 链表

微信公众号：王道在线

王道论坛网址：www.cskaoan.com

顺序表

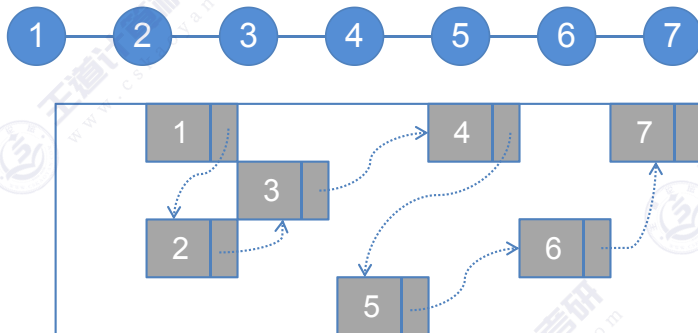
- 插入和删除操作移动大量元素。
- 数组的大小不好确定。
- 占用一大段连续的存储空间，造成很多碎片。

微信公众号：王道在线

王道论坛网址：www.cskaoan.com

单链表

data	next
------	------



逻辑上相邻的两个元素在物理位置上不相邻

单链表结点的定义：

```
typedef struct LNode{           //单链表结点类型
    ElemType data;              //数据域
    struct LNode *next;         //指针域
}LNode, *LinkList;
```

微信公众号：王道在线

王道论坛网址：www.cskaoan.com

单链表

头指针



头指针：链表中第一个结点的存储位置，用来标识单链表。

头结点：在单链表第一个结点之前附加的一个结点，为了操作上的方便。

若链表有头结点，则头指针永远指向头结点，不论链表是否为空，头指针均不为空，头指针是链表的必须元素，他标识一个链表。

头结点是为了操作的方便而设立的，其数据域一般为空，或者存放链表的长度。有头结点后，对在第一结点前插入和删除第一结点的操作就统一了，不需要频繁重置头指针。但头结点不是必须的。

微信公众号：王道在线

王道论坛网址：www.cskaoan.com

优缺点

优点

- 插入和删除操作不需要移动元素，只需要修改指针。
- 不需要大量的连续存储空间。

缺点

- 单链表附加指针域，也存在浪费存储空间的缺点。
- 查找操作时需要从表头开始遍历，依次查找，不能随机存取。

微信公众号：王道在线

王道论坛网址：www.cskaoan.com

插入操作

创建新结点代码：

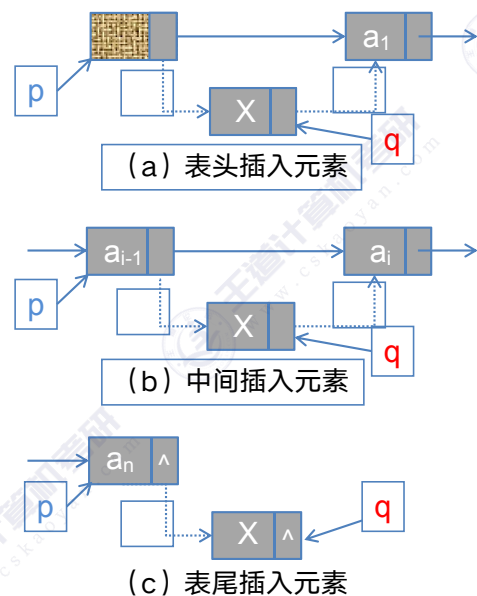
```
q = (LNode*) malloc(sizeof(LNode))
q->data = x;
```

(a) (b)操作的代码：

```
q->next = p->next;
p->next = q;
```

(c)操作的代码：

```
p->next = q;
q->next = NULL;
//请分别用头插法和尾插法创建一个单链表
```



微信公众号：王道在线

王道论坛网址：www.cskaoan.com

删除操作

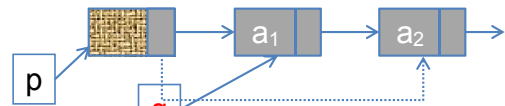
(a)(b)(c)操作的代码:

$p = \text{GetElem}(L, i-1); // \text{查找删除位置的前驱节点}$

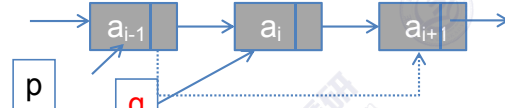
$q = p \rightarrow \text{next};$

$p \rightarrow \text{next} = q \rightarrow \text{next};$

$\text{free}(q);$



(a) 表头删除元素



(b) 中间删除元素



(c) 表尾删除元素

微信公众号：王道在线

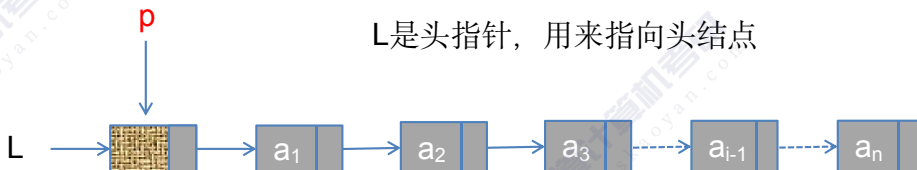
王道论坛网址：www.cskaoan.com

查找操作

按序号查找结点值的算法如下:

```
LNode *p=L->next;
int j=1;
while (p&& j<i) {
    p=p->next;
    j++;
}
return p;
```

L是头指针，用来指向头结点



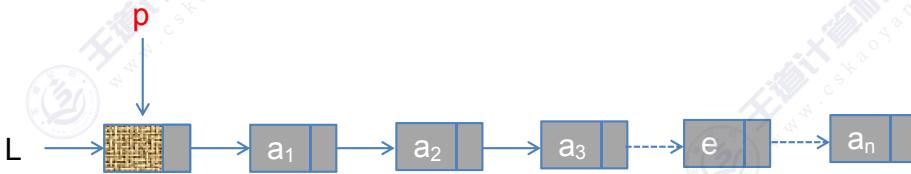
微信公众号：王道在线

王道论坛网址：www.cskaoan.com

查找
操作

按值查找结点的算法如下：

```
LNode *p=L->next;  
while (p!=NULL&& p->data!=e) {  
    p=p->next;  
}  
return p;
```



微信公众号：王道在线

王道论坛网址：www.cskao.com